

LABORATORIO 07

Mudarse sin cerrar el negocio

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo.

Introducción

1 · El tema

En este laboratorio vas a convertir una tabla Hive, como las que la DGT tiene hoy en producción, en una tabla Iceberg, sin copiar los datos y sin cambiarle el nombre. Y lo vas a hacer en los tres momentos de toda migración, preparar, convertir y verificar.

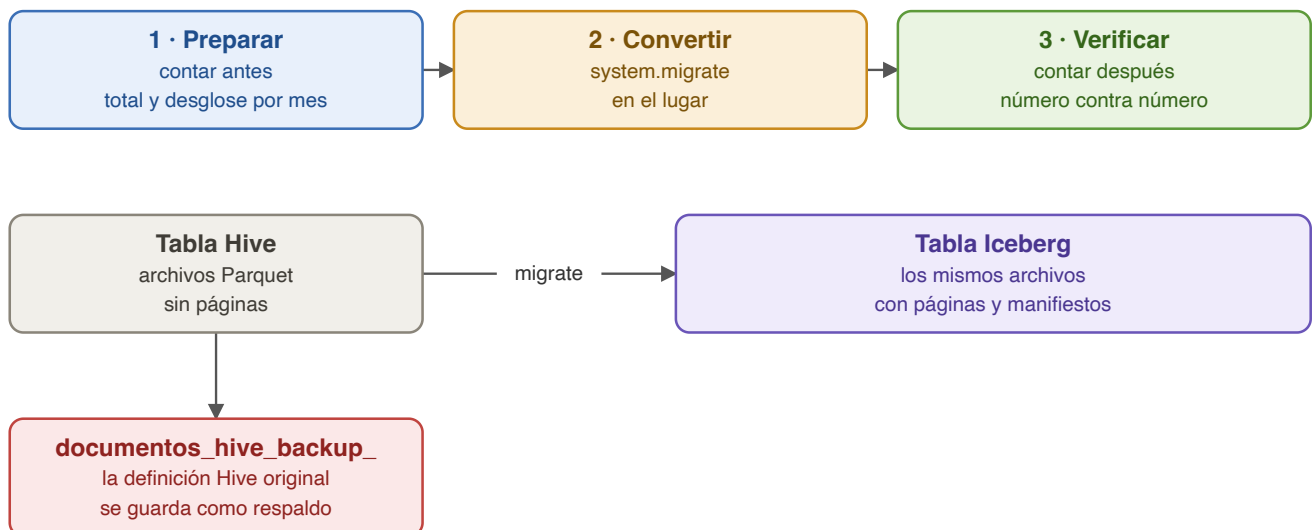
2 · El problema, en el almacén

El almacén se tiene que cambiar de local sin cerrar ni un día. Todo lo que hay en las estanterías tiene que llegar al local nuevo, y al final hay que poder demostrar que no se perdió ni un producto en el camino. Si el inventario de antes y el de después no dan lo mismo, no fue una mudanza, fue una pérdida.

3 · El problema en términos técnicos

Una tabla Hive guarda sus datos en archivos Parquet dentro de una carpeta, y el catálogo solo sabe dónde está esa carpeta. No tiene páginas, así que no se puede mirar cómo estaba ayer ni deshacer una carga. Pasarla a Iceberg copiando todo sería lento y caro con años de historia, y obligaría a cambiar el nombre y avisar a todos los que la consultan. Además, una migración sin verificación es solo una esperanza. Hay que medir antes y después.

4 · El diagrama



1 · Preparar (azul). Antes de tocar nada se levanta el inventario, cuántos documentos hay y cuánto suman, en total y mes por mes.

2 · Convertir (amarillo). El procedimiento `system.migrate` convierte la tabla en el lugar. No copia datos. Escribe los manifiestos y la primera página, y la tabla sigue con el mismo nombre.

3 · Verificar (verde). Se repiten las mismas mediciones y se comparan con las de antes. Si no dan lo mismo, la migración no está terminada.

Tabla Hive (gris). Es la tabla de partida, archivos Parquet en una carpeta y un catálogo que solo sabe dónde está esa carpeta.

Tabla Iceberg (morado). Son los mismos archivos, ahora nombrados en un manifiesto y colgados de una página. Desde ese momento la tabla tiene historia.

documentos_hive_backup_ (rojo). `migrate` guarda la definición Hive original con ese nombre. Los archivos de datos quedan bajo su carpeta, y por eso borrarla sin cuidado se lleva los datos de la tabla migrada.

5 · La solución

En el almacén, la mudanza empieza con el inventario completo del local viejo, producto por producto. Después se cambia el cartel y la forma de llevar las cuentas, sin mover las cajas de lugar. Y al final se hace el mismo inventario en el local nuevo y se compara, línea por línea.

En Iceberg eso es `CALL spark_catalog.system.migrate`. Reutiliza los archivos Parquet que ya existen, escribe los metadatos de Iceberg encima y conserva el nombre de la tabla. Antes y después se corren las mismas consultas de conteo, y los números tienen que coincidir al centavo.

6 · Los pasos

- **Paso 0.** Creas una tabla Hive de verdad, `documentos_hive`, con un año de documentos, y compruebas que es Hive.
- **Paso 1.** Levantas el inventario, el total y el desglose por mes.
- **Paso 2.** Ves sus límites. Pedirle las páginas o deshacer una carga falla.
- **Paso 3.** La migras con `system.migrate` y compruebas que ahora es Iceberg.
- **Paso 4.** Verificas con las mismas mediciones del paso 1.
- **Paso 5.** Pides las páginas y los archivos, que ahora sí existen, y miras las tablas de tu espacio.
- **Paso 6.** Consultas la tabla migrada desde Hue, otro motor, fuera del cuaderno.

Este laboratorio **no se puede repetir sin reiniciarlo**. Si ya lo corriste, antes corre el comando de abajo, desde la carpeta del repositorio. Borra las dos tablas y la carpeta del respaldo, que si no hacen fallar la creación de la tabla Hive.

```
bash
bin/reiniciar-lab.sh 07
```

Paso 0 · La tabla Hive de partida

Celda 0.1 · Entrar a tu espacio

La pregunta. ¿En qué espacio van a quedar las tablas?

Por qué ahora. La tabla Hive y su versión Iceberg se crean en tu espacio de trabajo, `mi_espacio`.

En el almacén. Es pararse en tu propio local antes de empezar la mudanza.

La sentencia, parte por parte.

- `%%sql` le dice al cuaderno que lo que sigue es SQL.
- `-- Celda 0.1` es un comentario que el motor ignora.
- `USE mi_espacio` deja a `mi_espacio` como espacio por omisión.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:07:51 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. La sentencia se ejecutó. Las tres líneas de arriba son avisos de arranque de Spark, no errores.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo avisos y errores.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar ese nivel, con `sc`, el contexto de Spark. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que no encontró una biblioteca de Hadoop compilada para este sistema y usa la de Java. `26/09/25 18:07:51` es la fecha y la hora del aviso, 25 de septiembre de 2026 a las 18:07:51 UTC.

Celda 0.2 · Borrar la tabla, si existe

La pregunta. ¿Cómo se parte sin la tabla de una corrida anterior?

Por qué ahora. Si `documentos_hive` ya existe, la creación falla.

En el almacén. Es vaciar el local antes de montar la estantería de prueba.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla.
- `IF EXISTS` evita el error si no existe.
- `documentos_hive` es la tabla, en `mi_espacio`.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS documentos_hive
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó.

Celda 0.3 · Borrar el respaldo, si existe

La pregunta. ¿Cómo se borra el respaldo de una migración anterior?

Por qué ahora. La migración crea una tabla `documentos_hive_backup_`. Si quedó de antes, la migración nueva falla.

En el almacén. Es sacar del local viejo el inventario de una mudanza pasada.

La sentencia, parte por parte.

- `DROP TABLE IF EXISTS` borra la tabla si existe.
- `documentos_hive_backup_` es el nombre que `migrate` le pone al respaldo, el nombre original con `_backup_` al final.

```
sql
%%sql
-- Celda 0.3
DROP TABLE IF EXISTS documentos_hive_backup_
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó. `DROP TABLE` no borra la carpeta de los archivos, y por eso para repetir el laboratorio hace falta `bin/reiniciar-lab.sh 07`.

Celda 0.4 · Crear la tabla Hive

La pregunta. ¿Cómo se declara una tabla Hive clásica?

Por qué ahora. Es el punto de partida, la clase de tabla que la DGT tiene hoy en su data lake.

En el almacén. Es montar el local viejo, con sus estanterías de siempre.

La sentencia, parte por parte.

- `CREATE TABLE documentos_hive` crea la tabla.
- Entre paréntesis van las columnas con su tipo. `STRING` es texto, `INT` entero, `BIGINT` entero grande, `DATE` fecha y `DECIMAL(18,2)` un número con 18 dígitos en total y 2 decimales, para montos.
- `rut_emisor` es quien emitió el documento, `tipo_dte` el tipo de documento tributario, `folio` su número, `fecha_emision` la fecha, `monto_neto`, `monto_iva` y `monto_total` los montos, y `estado_sii` en qué estado quedó.
- `STORED AS PARQUET` la declara como tabla Hive que guarda sus datos en Parquet. No dice `USING iceberg`, y eso es lo que la hace Hive.

```
sql
%%sql
-- Celda 0.4
CREATE TABLE documentos_hive (
  rut_emisor      STRING,
  tipo_dte        INT,
  folio           BIGINT,
  fecha_emision   DATE,
  monto_neto      DECIMAL(18,2),
  monto_iva       DECIMAL(18,2),
  monto_total     DECIMAL(18,2),
  estado_sii      STRING
) STORED AS PARQUET
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla Hive quedó creada, vacía.

Celda 0.5 · Cargarla con un año

La pregunta. ¿Cómo se llena la tabla con un año de documentos?

Por qué ahora. Una migración de verdad mueve una tabla con datos.

En el almacén. Es llenar las estanterías del local viejo.

La sentencia, parte por parte.

- `INSERT INTO documentos_hive` agrega filas a la tabla Hive.
- `SELECT rut_emisor, tipo_dte, folio, fecha_emision, monto_neto, monto_iva, monto_total, estado_sii` elige las ocho columnas, en el mismo orden de la tabla.
- `FROM curso.dte_2024` es un año de documentos de 2024 que trae el ambiente, en el espacio `curso`.

```
sql
%%sql
-- Celda 0.5
INSERT INTO documentos_hive
SELECT rut_emisor, tipo_dte, folio, fecha_emision,
       monto_neto, monto_iva, monto_total, estado_sii
FROM curso.dte_2024
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La carga entró.

Celda 0.6 · Qué clase de tabla es

La pregunta. ¿La tabla es Hive o Iceberg?

Por qué ahora. Antes de migrar hay que confirmar de dónde se parte.

En el almacén. Es leer el cartel del local viejo.

La sentencia, parte por parte.

- `DESCRIBE TABLE EXTENDED` pide la descripción completa de la tabla, columnas y detalles.
- `documentos_hive` es la tabla.

```
sql
%%sql
-- Celda 0.6
DESCRIBE TABLE EXTENDED documentos_hive
```

Lo que sale en pantalla.

```
text
col_name      data_type      comment
rut_emisor    string          None
tipo_dte      int             None
folio         bigint        None
fecha_emision date           None
monto_netto   decimal(18,2)   None
monto_iva     decimal(18,2)   None
monto_total   decimal(18,2)   None
estado_sii    string          None

# Detailed Table Information
Database      mi_espacio
Table         documentos_hive
Owner         root
Created Time   Fri Sep 25 18:07:55 UTC 2026
Last Access    UNKNOWN
Created By     Spark 3.3.4
Type          MANAGED
Provider       hive
Table Properties [transient_lastDdlTime=1790359675]
Location      hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/documentos_hive
Serde Library  org.apache.hadoop.hive ql.io.parquet.serde.ParquetHiveSerDe
InputFormat    org.apache.hadoop.hive ql.io.parquet.MapredParquetInputFormat
OutputFormat   org.apache.hadoop.hive ql.io.parquet.MapredParquetOutputFormat
Storage Properties [serialization.format=1]
Partition Provider Catalog
25 filas.
```

Cómo se lee. La línea que importa es `Provider`, que dice `hive`. Es una tabla Hive corriente.

Las otras líneas, de arriba abajo.

- Las ocho primeras son las columnas con su tipo. `None` en `comment` significa que no tienen comentario.
- `# Detailed Table Information` separa los detalles.
- `Database` es el espacio, `mi_espacio`, y `Table` el nombre.
- `Owner` es el usuario que la creó, `root`, que es el usuario del contenedor.
- `Created Time` es cuándo se creó, en UTC, y `Last Access` el último acceso, que Hive no registra.
- `Created By` es el motor que la creó, `Spark 3.3.4`.
- `Type` dice `MANAGED`, que significa que el catálogo administra la carpeta de los datos.
- `Table Properties` trae `transient_lastDdlTime`, la hora del último cambio de estructura, en segundos desde 1970.
- `Location` es la carpeta en HDFS donde viven los archivos.
- `Serde Library` es `ParquetHiveSerDe`, la pieza que Hive usa para leer y escribir Parquet. `InputFormat` y `OutputFormat` son las clases que leen y escriben los archivos.
- `Storage Properties` y `Partition Provider` son detalles del almacenamiento. `Catalog` indica que las particiones las lleva el catálogo, aunque esta tabla no tiene.

Paso 1 · Levantar el inventario

Celda 1.1 · El total

La pregunta. ¿Cuántos documentos hay y cuánto suman, antes de migrar?

Por qué ahora. Una migración se verifica contra lo que había, y lo que había solo se sabe si se mide antes de tocar nada.

En el almacén. Es contar todos los productos del local viejo y sumar su valor.

La sentencia, parte por parte.

- `count(*)` AS `documentos` cuenta las filas.
- `sum(monto_total)` AS `total_general` suma los montos.
- `FROM documentos_hive` es la tabla Hive.

```
sql
%%sql
-- Celda 1.1
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_general
FROM documentos_hive
```

Lo que sale en pantalla.

```
text
documentos  total_general
30000       291293351462.71
1 fila.
```

Cómo se lee. 30000 documentos que suman 291.293.351.462,71 pesos. Son los dos números contra los que se va a comparar.

Celda 1.2 · El desglose por mes

La pregunta. ¿Cuántos documentos y cuánto monto hay en cada mes?

Por qué ahora. Si algo se pierde en la mudanza, el desglose dice en qué mes se perdió.

En el almacén. Es contar estantería por estantería.

La sentencia, parte por parte.

- `date_format(fecha_emision, 'yyyy-MM')` AS `periodo` convierte la fecha en el texto del año y el mes, por ejemplo `2024-06`.
- `count(*)` AS `documentos` cuenta los documentos de cada mes.
- `sum(monto_total)` AS `total_del_periodo` suma sus montos.
- `FROM documentos_hive` es la tabla Hive.
- `GROUP BY date_format(fecha_emision, 'yyyy-MM')` agrupa las filas por mes.
- `ORDER BY periodo` ordena de enero a diciembre.

```

sql
%%sql
-- Celda 1.2
SELECT date_format(fecha_emision, 'yyyy-MM') AS periodo,
       count(*) AS documentos,
       sum(monto_total) AS total_del_periodo
FROM documentos_hive
GROUP BY date_format(fecha_emision, 'yyyy-MM')
ORDER BY periodo

```

Lo que sale en pantalla.

```

text
periodo documentos total_del_periodo
2024-01 2500 23381729910.07
2024-02 2500 24831948613.59
2024-03 2500 26699931432.64
2024-04 2500 23722997990.55
2024-05 2500 24328007216.84
2024-06 2500 24547341762.59
2024-07 2500 23621609483.83
2024-08 2500 23648615665.89
2024-09 2500 24136964210.51
2024-10 2500 23106058024.78
2024-11 2500 24722608269.83
2024-12 2500 24545538881.59
12 filas.

```

Cómo se lee. Doce meses de 2024, con 2500 documentos cada uno. Junio, por ejemplo, suma 24547341762.59. Ese es el inventario.

Paso 2 · Mirar sus límites

Celda 2.1 · Pedirle las páginas

La pregunta. ¿Una tabla Hive tiene páginas?

Por qué ahora. Antes de mudarse, conviene ver qué le falta a la tabla vieja. Esta celda falla a propósito.

En el almacén. Es pedirle al local viejo su libreta de páginas fechadas, y descubrir que no tiene.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide el número de página, la hora y la operación.
- `FROM mi_espacio.documentos_hive.snapshots` intenta leer la vista `.snapshots`, que solo existe en tablas Iceberg.
- `ORDER BY committed_at` ordena por hora.

```
sql
%%sql
-- Celda 2.1
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.documentos_hive.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
Spark rechazó la sentencia:
spark_catalog requires a single-part namespace, but got [mi_espacio, documentos_hive]
```

Cómo se lee. Spark rechazó la sentencia. Como `documentos_hive` no es Iceberg, Spark no entiende `.snapshots` como una vista y lee el nombre como un espacio de dos partes, `mi_espacio` y `documentos_hive`. `spark_catalog requires a single-part namespace` dice que el catálogo solo acepta espacios de una parte. En resumen, la tabla no tiene páginas.

Celda 2.2 · Intentar deshacer una carga

La pregunta. ¿Se puede volver una tabla Hive a una página anterior?

Por qué ahora. Es la otra cosa que la tabla vieja no puede hacer. También falla a propósito, y es la última que falla.

En el almacén. Es pedirle al local viejo que vuelva a como estaba ayer.

La sentencia, parte por parte.

- `CALL spark_catalog.system.rollback_to_snapshot` es el procedimiento de Iceberg que vuelve una tabla a una página anterior.
- `'mi_espacio.documentos_hive'` es la tabla.
- `1` es el número de página. Da lo mismo cuál se ponga, porque el procedimiento falla antes de mirarlo.

```

sql
%%sql
-- Celda 2.2
CALL spark_catalog.system.rollback_to_snapshot('mi_espacio.documentos_hive', 1)

```

Lo que sale en pantalla.

```

text
Spark rechazó la sentencia:

An error occurred while calling o39.sql.
: org.apache.iceberg.exceptions.ValidationException: mi_espacio.documentos_hive is not
org.apache.iceberg.spark.source.SparkTable
    at org.apache.iceberg.exceptions.ValidationException.check(ValidationException.java:49)
    at
org.apache.iceberg.spark.procedures.BaseProcedure.loadSparkTable(BaseProcedure.java:141)
    at org.apache.iceberg.spark.procedures.BaseProcedure.execute(BaseProcedure.java:101)
    at
org.apache.iceberg.spark.procedures.BaseProcedure.modifyIcebergTable(BaseProcedure.java:85)
    at
org.apache.iceberg.spark.procedures.RollbackToSnapshotProcedure.call(RollbackToSnapshotProcedure.
java:83)
    at org.apache.spark.sql.execution.datasources.v2.CallExec.run(CallExec.scala:34)
    at
org.apache.spark.sql.execution.datasources.v2.V2CommandExec.result$lzycompute(V2CommandExec.scala
:43)
    at
org.apache.spark.sql.execution.datasources.v2.V2CommandExec.result(V2CommandExec.scala:43)
    at
org.apache.spark.sql.execution.datasources.v2.V2CommandExec.executeCollect(V2CommandExec.scala:49
)
    at
org.apache.spark.sql.execution.QueryExecution$$anonfun$eagerlyExecuteCommands$1.$anonfun$applyOrElse$1(QueryExecution.scala:98)
    at
org.apache.spark.sql.execution.SQLExecution$.withNewExecutionId(SQLExecution.scala:109)
    at
org.apache.spark.sql.execution.SQLExecution$.withSQLConfPropagated(SQLExecution.scala:169)
    at
org.apache.spark.sql.execution.SQLExecution$.withNewExecutionId(SQLExecution.scala:95)
    at org.apache.spark.sql.SparkSession.withActive(SparkSession.scala:779)
    at org.apache.spark.sql.execution.SQLExecution$.withNewExecutionId(SQLExecution.scala:64)
    at
org.apache.spark.sql.execution.QueryExecution$$anonfun$eagerlyExecuteCommands$1.applyOrElse(Query
Execution.scala:98)
    at
org.apache.spark.sql.execution.QueryExecution$$anonfun$eagerlyExecuteCommands$1.applyOrElse(Query
Execution.scala:94)
    at
org.apache.spark.sql.catalyst.trees.TreeNode.$anonfun$transformDownWithPruning$1(TreeNode.scala:5
84)
    at org.apache.spark.sql.catalyst.trees.CurrentOrigin$.withOrigin(TreeNode.scala:176)
    at
org.apache.spark.sql.catalyst.trees.TreeNode.transformDownWithPruning(TreeNode.scala:584)
    at
org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.org$apache$spark$sql$catalyst$plans$logi
cal$AnalysisHelper$$super$transformDownWithPruning(LogicalPlan.scala:30)
    at
org.apache.spark.sql.catalyst.plans.logical.AnalysisHelper.transformDownWithPruning(AnalysisHelpe
r.scala:267)
    at

```

```

org.apache.spark.sql.catalyst.plans.logical.AnalysisHelper.transformDownWithPruning$(AnalysisHelp
er.scala:263)
    at
org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.transformDownWithPruning(LogicalPlan.scal
a:30)
    at
org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.transformDownWithPruning(LogicalPlan.scal
a:30)
        at org.apache.spark.sql.catalyst.trees.TreeNode.transformDown(TreeNode.scala:560)
        at
org.apache.spark.sql.execution.QueryExecution.eagerlyExecuteCommands(QueryExecution.scala:94)
    at
org.apache.spark.sql.execution.QueryExecution.commandExecuted$lzycompute(QueryExecution.scala:81)
    at org.apache.spark.sql.execution.QueryExecution.commandExecuted(QueryExecution.scala:79)
    at org.apache.spark.sql.Dataset.<init>(Dataset.scala:219)
    at org.apache.spark.sql.Dataset$.anonfun$ofRows$2(Dataset.scala:99)
    at org.apache.spark.sql.SparkSession.withActive(SparkSession.scala:779)
    at org.apache.spark.sql.Dataset$.ofRows(Dataset.scala:96)
    at org.apache.spark.sql.SparkSession$.anonfun$sql$1(SparkSession.scala:622)
    at org.apache.spark.sql.SparkSession.withActive(SparkSession.scala:779)
    at org.apache.spark.sql.SparkSession.sql(SparkSession.scala:617)
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at
java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
    at
java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.j
ava:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:566)
        at py4j.reflection.MethodInvoker.invoke(MethodInvoker.java:244)
        at py4j.reflection.ReflectionEngine.invoke(ReflectionEngine.java:357)
        at py4j.Gateway.invoke(Gateway.java:282)
        at py4j.commands.AbstractCommand.invokeMethod(AbstractCommand.java:132)
        at py4j.commands.CallCommand.execute(CallCommand.java:79)
        at py4j.ClientServerConnection.waitForCommands(ClientServerConnection.java:182)
        at py4j.ClientServerConnection.run(ClientServerConnection.java:106)
        at java.base/java.lang.Thread.run(Thread.java:829)

```

Cómo se lee. Lo que importa está en la tercera línea, `ValidationException: mi_espacio.documentos_hive is not org.apache.iceberg.spark.source.SparkTable`. Dice que la tabla no es una tabla Iceberg, así que no hay nada que revertir. Ni siquiera llegó a mirar el número.

El resto es la traza de Java, la lista de funciones que se estaban ejecutando cuando ocurrió el error, de la más interna a la más externa. Cada línea `at` nombra una función y el archivo y la línea donde está. Sirve para quien programa Iceberg o Spark, no para quien lo usa. `An error occurred while calling o39.sql` es la forma en que Python informa el error, y `o39.sql` es el nombre interno de la llamada. Las líneas con `py4j` son el puente entre Python y Java.

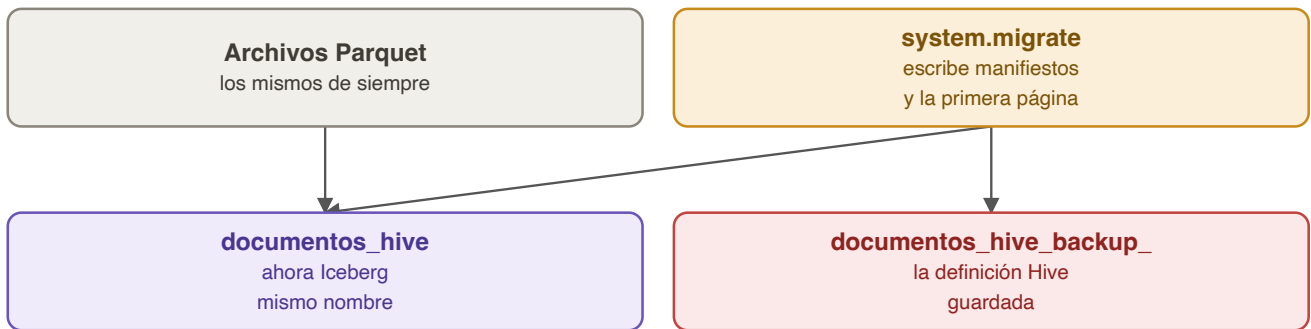
Paso 3 · Migrar

Celda 3.1 · La migración

La pregunta. ¿Cómo se convierte la tabla Hive en Iceberg sin copiar los datos?

Por qué ahora. Con el inventario hecho, ya se puede mudar.

En el almacén. Es cambiar el cartel y la forma de llevar las cuentas, dejando las cajas donde están.



Archivos Parquet (gris). Los archivos de datos no se copian ni se reescriben. Se reutilizan tal cual.

system.migrate (amarillo). Escribe los metadatos de Iceberg, el manifiesto que nombra los archivos y la primera página.

documentos_hive (morado). La tabla conserva el nombre, así que las consultas de siempre siguen funcionando.

documentos_hive_backup_ (rojo). La definición Hive original queda guardada con ese nombre, como respaldo.

La sentencia, parte por parte.

- `CALL` ejecuta un procedimiento.
- `spark_catalog.system.migrate` es el procedimiento de Iceberg que convierte una tabla en el lugar.
- `'mi_espacio.documentos_hive'` es la tabla, entre comillas y con su espacio.

```
sql
%%sql
-- Celda 3.1
CALL spark_catalog.system.migrate('mi_espacio.documentos_hive')
```

Lo que sale en pantalla.

```
text
migrated_files_count
1
1 fila.
```

Cómo se lee. `migrated_files_count` es 1. La tabla tenía un solo archivo de datos, y quedó incorporado.

Celda 3.2 · Qué clase de tabla es ahora

La pregunta. ¿La tabla ya es Iceberg?

Por qué ahora. Es la misma pregunta de la celda 0.6, para comparar.

En el almacén. Es leer el cartel nuevo.

La sentencia, parte por parte.

- `DESCRIBE TABLE EXTENDED` pide la descripción completa.
- `documentos_hive` es la misma tabla, con el mismo nombre.

```
sql
%%sql
-- Celda 3.2
DESCRIBE TABLE EXTENDED documentos_hive
```

Lo que sale en pantalla.

```
text
col_name      data_type
comment
rut_emisor    string
tipo_dte      int
folio         bigint
fecha_emision date
monto_neto    decimal(18,2)
monto_iva     decimal(18,2)
monto_total   decimal(18,2)
estado_sii    string

# Partitioning
Not partitioned

# Metadata Columns
_spec_id      int
_partition    struct<
_file         string
_pos         bigint
_deleted      boolean

# Detailed Table Information
Name          spark_catalog.mi_espacio.documentos_hive
Location      hdfs://namenode:8020/warehouse/iceberg/mi_espacio.db/documentos_hive
Provider      iceberg
Table Properties
current-snapshot-id=7041860744555628170,format=iceberg/parquet,format-version=1,migrated=true,schema.name-mapping.default=[{"field-id":1,"names":["rut_emisor"]}, {"field-id":2,"names":["tipo_dte"]}, {"field-id":3,"names":["folio"]}, {"field-id":4,"names":["fecha_emision"]}, {"field-id":5,"names":["monto_neto"]}, {"field-id":6,"names":["monto_iva"]}, {"field-id":7,"names":["monto_total"]}, {"field-id":8,"names":["estado_sii"]}
]
24 filas.
```

Cómo se lee. `Provider` ahora dice `iceberg`. Misma tabla, mismo nombre, otra naturaleza.

El resto de las líneas.

- Las ocho columnas son las mismas.
- `# Partitioning` y `Not partitioned` dicen que la tabla no tiene cajones.
- `# Metadata Columns` son columnas ocultas que Iceberg agrega. `_spec_id` es el criterio de partición de cada fila, `_partition` su cajón, `_file` el papelito donde vive, `_pos` su posición dentro del papelito y `_deleted` si está borrada.
- `Name` es el nombre completo, `spark_catalog.mi_espacio.documentos_hive`, y `Location` la carpeta.
- `Table Properties` trae `current-snapshot-id`, la página vigente, `7041860744555628170`, `format=iceberg/parquet`, `format-version=1`, `migrated=true`, que marca que la tabla nació de una migración, y `schema.name-mapping.default`, que le dice a Iceberg qué columna de los archivos viejos corresponde a cada campo, por nombre. Los `\n` son saltos de línea escritos como texto.

La fila de títulos sale muy ancha porque el cuaderno alinea las columnas según el valor más largo, que es esa propiedad.

Paso 4 · Verificar

Celda 4.1 · El total, otra vez

La pregunta. ¿El total es el mismo que antes de migrar?

Por qué ahora. Es el paso que la gente omite, y el que separa una migración profesional de un susto.

En el almacén. Es contar de nuevo todos los productos, ahora en el local nuevo.

La sentencia, parte por parte. Es la misma de la celda 1.1, letra por letra.

- `count(*) AS documentos` cuenta las filas.
- `sum(monto_total) AS total_general` suma los montos.
- `FROM documentos_hive` es la tabla, que ahora es Iceberg.

```
sql
%%sql
-- Celda 4.1
SELECT count(*)          AS documentos,
       sum(monto_total) AS total_general
FROM documentos_hive
```

Lo que sale en pantalla.

```
text
documentos  total_general
30000       291293351462.71
1 fila.
```

Cómo se lee. 30000 documentos y 291.293.351.462,71 pesos. El mismo conteo y el mismo total de la celda 1.1, al centavo.

Celda 4.2 · El desglose, otra vez

La pregunta. ¿Cada mes quedó igual?

Por qué ahora. El total podría coincidir por casualidad. Mes por mes no.

En el almacén. Es contar estantería por estantería en el local nuevo.

La sentencia, parte por parte. Es la misma de la celda 1.2.

- `date_format(fecha_emision, 'yyyy-MM')` AS periodo convierte la fecha en año y mes.
- `count(*) AS documentos` y `sum(monto_total) AS total_del_periodo` cuentan y suman por mes.
- `GROUP BY` agrupa por mes y `ORDER BY periodo` ordena.

```

sql
%%sql
-- Celda 4.2
SELECT date_format(fecha_emision, 'yyyy-MM') AS periodo,
       count(*) AS documentos,
       sum(monto_total) AS total_del_periodo
FROM documentos_hive
GROUP BY date_format(fecha_emision, 'yyyy-MM')
ORDER BY periodo

```

Lo que sale en pantalla.

```

text
periodo documentos total_del_periodo
2024-01 2500 23381729910.07
2024-02 2500 24831948613.59
2024-03 2500 26699931432.64
2024-04 2500 23722997990.55
2024-05 2500 24328007216.84
2024-06 2500 24547341762.59
2024-07 2500 23621609483.83
2024-08 2500 23648615665.89
2024-09 2500 24136964210.51
2024-10 2500 23106058024.78
2024-11 2500 24722608269.83
2024-12 2500 24545538881.59
12 filas.

```

Cómo se lee. Los mismos doce meses, con los mismos 2500 documentos y los mismos totales de la celda 1.2. La mudanza no perdió nada, y está comprobado.

Paso 5 · Comprobar que ahora sí es Iceberg

Celda 5.1 · Las páginas

La pregunta. ¿La tabla migrada tiene páginas?

Por qué ahora. Es la misma consulta que falló en la celda 2.1.

En el almacén. Es pedirle al local nuevo su libreta de páginas.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide número de página, hora y operación.
- `FROM mi_espacio.documentos_hive.snapshots` lee la vista de páginas.
- `ORDER BY committed_at` ordena por hora.

```
sql
%%sql
-- Celda 5.1
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.documentos_hive.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
7041860744555628170  2026-09-25 18:08:02.285  append
1 fila.
```

Cómo se lee. Una página, `7041860744555628170`, `append`, de las 18:08:02 UTC, las 15:08 en Chile. Es la que escribió la migración. De aquí en adelante cada carga y cada corrección van a dejar la suya.

Celda 5.2 · Los archivos

La pregunta. ¿Cuántos archivos tiene la tabla migrada?

Por qué ahora. Para ver que no se copió nada.

En el almacén. Es contar las cajas del local nuevo.

La sentencia, parte por parte.

- `count(*) AS archivos` cuenta los papelitos.
- `sum(record_count) AS documentos` suma las filas de cada uno. `record_count` es cuántas filas trae un papelito.
- `FROM mi_espacio.documentos_hive.files` es la vista de los papelitos vigentes.

```
sql
%%sql
-- Celda 5.2
SELECT count(*)          AS archivos,
       sum(record_count) AS documentos
FROM mi_espacio.documentos_hive.files
```

Lo que sale en pantalla.

```
text
archivos documentos
1 30000
1 fila.
```

Cómo se lee. 1 archivo con los 30000 documentos. Es el mismo archivo Parquet de siempre. Solo se le pusieron manifiestos encima.

Celda 5.3 · Las tablas del espacio

La pregunta. ¿Qué tablas hay ahora en tu espacio?

Por qué ahora. Para encontrar el respaldo que dejó la migración.

En el almacén. Es recorrer el local y ver qué quedó en cada estantería.

La sentencia, parte por parte.

- `SHOW TABLES IN mi_espacio` lista las tablas del espacio.

```
sql
%%sql
-- Celda 5.3
SHOW TABLES IN mi_espacio
```

Lo que sale en pantalla.

```
text
namespace tableName isTemporary
mi_espacio contribuyentes_lab00 False
mi_espacio contribuyentes_lab01 False
mi_espacio contribuyentes_lab02 False
mi_espacio documentos_hive False
mi_espacio documentos_hive_backup_ False
mi_espacio documentos_lab03 False
mi_espacio documentos_lab05 False
mi_espacio documentos_por_mes False
mi_espacio documentos_sin_particion False
mi_espacio recepcion_lab06 False
10 filas.
```

Cómo se lee. Están `documentos_hive`, la migrada, y `documentos_hive_backup_`, el respaldo. Las demás son tablas que ya había en tu espacio, y en tu pantalla pueden ser otras. `namespace` es el espacio, `tableName` el nombre y `isTemporary` dice si es una vista temporal, que aquí ninguna es.

Paso 6 · La capa de consulta

Este paso no va en el cuaderno. Se hace en Hue, en el navegador, que es por donde se conectan las herramientas de reportería. Hue es otro motor, el Spark Thrift Server, que lee la misma tabla por el mismo catálogo.

Para usarlo, el ambiente tiene que estar arriba con Hue.

```
bash
bin/ambiente.sh arriba --sql
```

Abre <http://localhost:8889/> en otra pestaña, entra con tu usuario y abre el editor **SparkSQL (Iceberg)**. En Hue el nombre de la tabla va completo, con el espacio adelante.

La primera consulta es el desglose por mes.

```
sql
SELECT date_format(fecha_emision, 'yyyy-MM') AS periodo,
       count(*) AS documentos,
       sum(monto_total) AS total_del_periodo
FROM mi_espacio.documentos_hive
GROUP BY date_format(fecha_emision, 'yyyy-MM')
ORDER BY periodo
```

Da exactamente lo mismo que la celda 4.2, los doce meses con los mismos totales. Otro motor, la misma tabla.

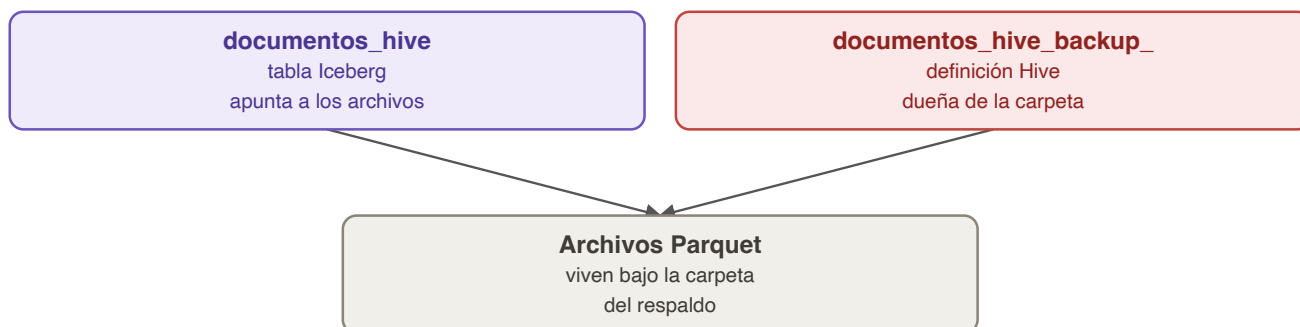
La segunda agrupa por tipo de documento.

```
sql
SELECT tipo_dte,
       count(*) AS documentos,
       sum(monto_total) AS total
FROM mi_espacio.documentos_hive
GROUP BY tipo_dte
ORDER BY tipo_dte
```

Da seis filas, una por tipo de documento. El tipo 33 es la factura afecta y el que más documentos tiene. El 61 es la nota de crédito, y su total es negativo porque resta.

El respaldo

`documentos_hive_backup_` guarda la definición Hive original, y aquí va la advertencia más importante del laboratorio.



documentos_hive (morado). La tabla migrada nombra los archivos en su manifiesto.

documentos_hive_backup_ (rojo). El respaldo es el dueño de la carpeta donde físicamente están esos archivos.

Archivos Parquet (gris). Los mismos archivos para las dos. Si alguien borra el respaldo borrando también sus datos, se lleva los datos de la tabla migrada.

El respaldo se conserva mientras dura la validación. Cuando se decida eliminarlo, se hace comprobando antes dónde están los archivos, con la consulta de `.files`.

Preguntas frecuentes

¿Por qué no se copiaron los datos?

Porque `migrate` reutiliza los archivos Parquet que ya existen. Lo único que escribe son metadatos, el manifiesto que nombra los archivos y la primera página. Por eso demora lo mismo con una tabla chica que con una de años de historia.

¿Hay otras formas de migrar?

Sí. `system.snapshot` crea una tabla Iceberg nueva sobre los mismos archivos sin tocar la original, y sirve para probar sin riesgo. `CREATE TABLE ... USING iceberg AS SELECT ...` copia los datos a una tabla nueva, y sirve si además quieres cambiar el esquema o particionar distinto. `system.add_files` suma archivos que ya existen a una tabla Iceberg creada. El orden recomendado es probar primero con `snapshot` y migrar después con `migrate`.

¿Por qué hay que reiniciar para repetir el laboratorio?

Porque `DROP TABLE` borra la definición pero no la carpeta. Si la carpeta queda, crear otra vez la tabla Hive en el mismo lugar falla. `bin/reiniciar-lab.sh 07` borra las tablas y también las carpetas.

¿Qué pasa si hay procesos escribiendo durante la migración?

Hay que detenerlos. `migrate` no cierra el negocio, pero tampoco atiende a dos cajeros a la vez. Una carga que escribe en la tabla Hive mientras se convierte puede quedar fuera del manifiesto.

¿Esto funciona igual en la plataforma de la DGT?

La lógica es la misma, pero este ambiente no tiene Kerberos ni Ranger. `migrate` renombra la tabla original, y renombrar exige permisos sobre el catálogo que pueden no estar dados. Si falla por permisos, `system.snapshot` no renombra nada y suele pasar donde el otro no.

¿Qué significa None en la descripción de la tabla Hive?

Que la columna no tiene comentario. En la tabla migrada esa columna sale vacía en vez de `None`, porque Iceberg la describe de otra forma.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Una migración tiene tres momentos, y hiciste los tres. Contaste antes, convertiste y contaste después.

En el almacén. Te mudaste de local sin cerrar, sin mover las cajas, y con el inventario de antes y el de después dando exactamente lo mismo.

Los números de la solución ejecutada.

Medición	Antes, tabla Hive	Después, tabla Iceberg
Documentos	30000	30000
Total general	291293351462.71	291293351462.71
Meses con 2500 documentos	12	12
Páginas	no tiene	1
Archivos de datos	1	1

Momento Lo que hiciste

Preparar Contaste el total, el monto y el desglose por mes

Convertir `CALL spark_catalog.system.migrate('mi_espacio.documentos_hive')`

Verificar Repetiste las mismas mediciones y comparaste número contra número

Si alguno de los números no hubiera coincidido, la migración no estaría terminada, y lo sabrías ahora y no cuando alguien reclame por un reporte.